

5/1

DASS

→ Software Engineering X
Systems Engineering ✓

Tip 1 : Learn COBOL

Tip 2 : Learn non-CS courses
(Central University)

→ Safety - Critical Systems
↳ No Exploratory programming

→ Open-source / Real-world products :

- Big Userbase
- Millions of lines of code
- Effort
- Time
- Cost
- ★ • Specifications from stakeholders

→ Large Systems :

- B2C or B2B
↳ Customer ↳ Business

Systems :
{ Green field : Scratch }
{ Brown field : Existing }

→ Important :

- Design
- Teamwork
- Communication
- Programming
- Process

8/1

→ Sketch vs. Implementation :

Conformance ↳ Dynamic (Run-time) : Testing

① Feasibility Analysis

② Requirements → What ?

↕ Modeling

③ Design → How ?

④ Implementation

⑤ Test - Conformance checks , e.g. Pylint in Python

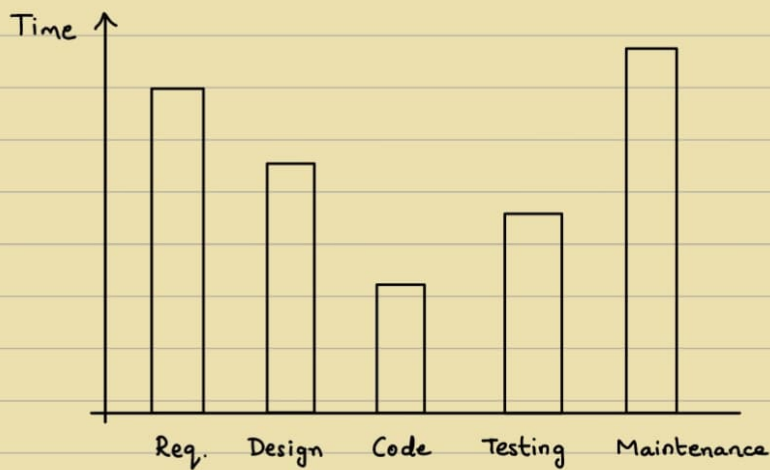
↳ Static testing & Dynamic Testing

⑥ Deployment

⑦ Maintenance

Development Lifecycle

SDLC : Software
Development
Life cycle



12/1

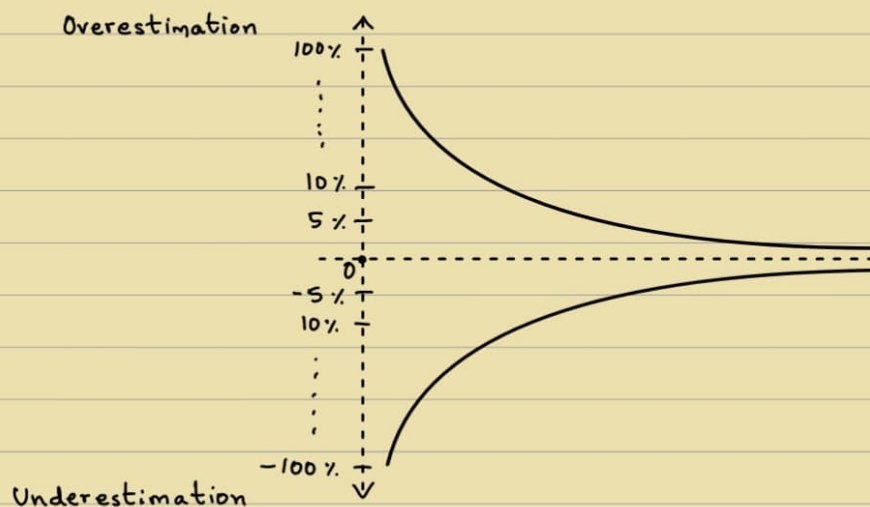
Traditional Lifecycle models

- Traditional Processes
- Highly Plan driven

Agile Models

- Agile processes
- Dynamically changes
- Short term Iteration

→ Cone of Uncertainty :



Feasibility :

- Planning
- Estimation
- Tracking

QSDT



- Rework → Technical Debt.
- PDCA → PDSA : Cycle of continuous Improvement
 - PDCA: Plan → Do → Check → Act
 - PDSA: Plan → Do → Study → Act

Plan : Planning , Estimation , Requirements , Design

Do : Implementation

Check : Testing and Deployment

Act : Retrospective (Maintenance + Documentation changes)

19/1

→ SDLC : Deming's cycle

① Feasibility Study :

- Technical Feasibility
- Operational Feasibility
- Schedule Feasibility
- Legal Feasibility
- Expensive Feasibility
- Planning , Estimation , Tracking , Artifacts

② Requirement Engineering :

- What client wants
- Technical , Business , Functional , Non-functional , Marketing Purposes
- Specification document
- SRS (Software Requirement Specifications)
- PRD (Product Requirement Document)

③ Design :

- High level design - Software Architecture [Artifact , Design , Document (ADD)]
- Low level design - Detailed design [Design document , e.g. UML (Interface)]

④ Implementation / Code :

- Code in the same programming language
- Frontend + Backend

⑤ Testing :

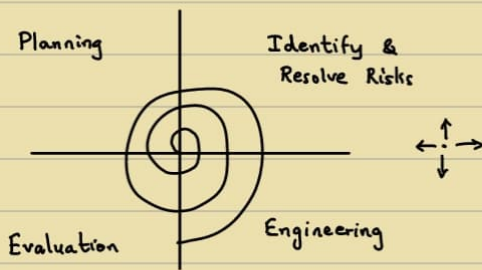
- Test plan
 - Reflects Implementation/code , Design & Requirements
- Test Oracle
 - Unit Testing
 - Integration Testing , System Testing (Low-level)
 - Acceptance Testing (High-level)

⑥ Maintenance :

- Preventive Maintenance - Defect Prevention
- Corrective Maintenance - Bug fixes
- Perfective Maintenance - Improvements / Enhancements
- Adaptive Maintenance - Switching

→ Process Models :

- Traditional Models (Plan driven)
- Waterfall Model
 - Finish one phase before going to the next one.
 - Sequential process
 - Documentation Heavy
 - Used when Requirements are stable and static
 - E.g. Banking Systems, Aircraft, Medical
- Prototyping Models :
 - Requirements are unclear
 - High user involvement
 - Focus on UI/UX & Not backend Heavy
- Evolutionary process Models : (For large problems)
(Iterative Incremental model)
 - Every Iteration (subsystem) is a waterfall. ← Speedy
- Spiral Model :



Iteration : Spiral
Meta-Model

∴ Every loop might be a process model in itself.

- When Safety is critical
- Project is not simple
- Budget is available abundantly.

22/1

→ Agile Process Models :

- User story : What the system does from user's perspective.
- Scrum Process
- Refactoring :


```
def calc_cost(distance, weight):
    standard_shipping_cost = (distance)(weight)
    return standard_shipping_cost
```

express = 0

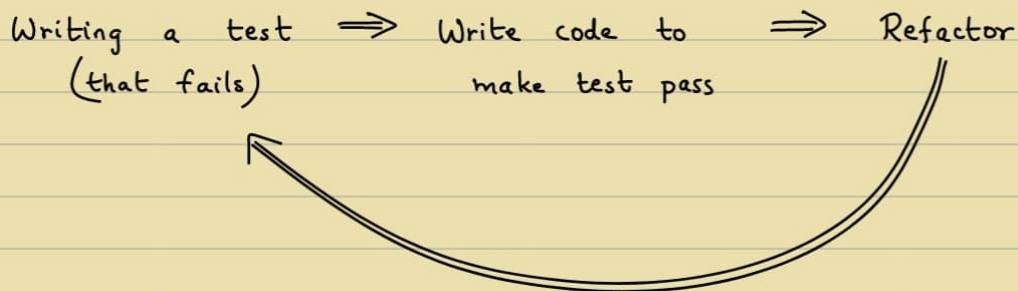
overnight = 0

international = 0

```
def calc_cost(distance, weight):
    standard_shipping_cost = (distance)(weight)
def handler(express, overnight, international, distance, weight):
    if (express == 1):
        return calc_cost(distance * 1.5, weight)
    if (overnight == 1):
        return calc_cost(distance * 3, weight)
    if (international == 1):
        return calc_cost(distance + fee, weight)
```

- Dead Code
- Open - Closed principle
- Technical Debt - Code smells
- Long Method - Single Responsibility principle
∴ Modularize - Method closes & ∴ Extendable
- System's complexity increases

- Test - Driven Development:



- Machine ↔ Machine working
- Continuous Integration (CI/CD)
Commit ⇒ Build ⇒ Test ⇒ Feedback
(Maven, Ant)

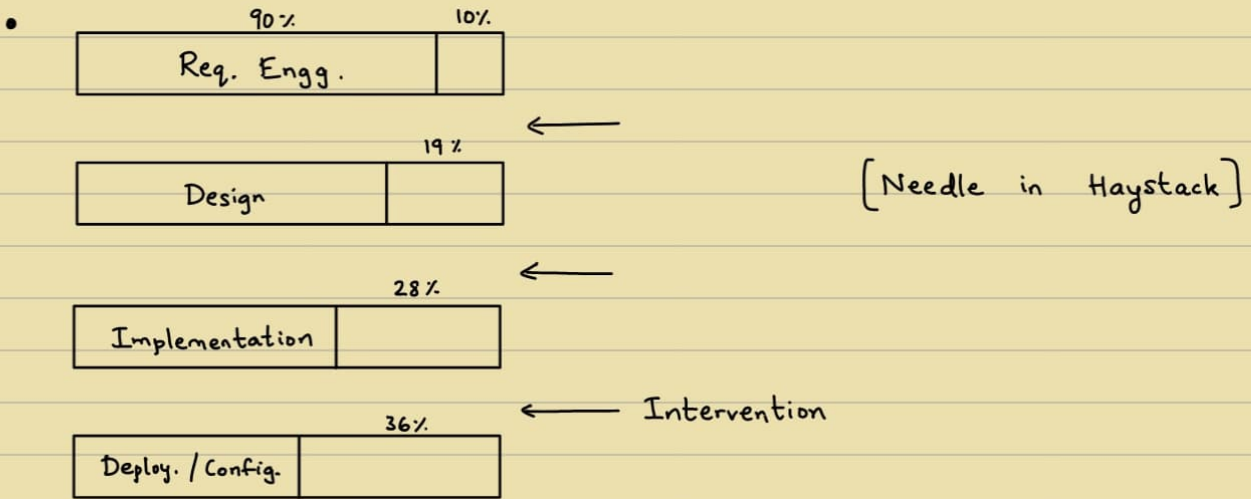
CD:

Manual: Delivery
Automatic: Deployment
(DevOps)

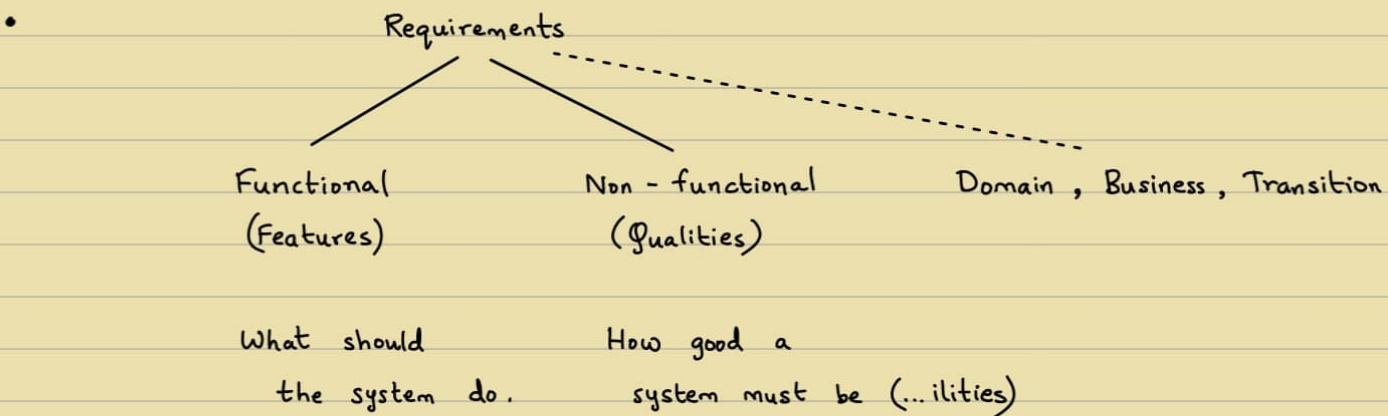
2/1

→ Requirement Engineering :

Computer Science and Engineering $\Rightarrow$ Software Engineering



- Requirements - Documented in SRS
- Safety Critical Systems - Closed System - Easy to Req. Engg.
Open Systems - Hard to Req. Engg.
- Say "NO" - Scope Creep
- Root-Cause of the mistake.



Functional requirements - Clearly defined - No Scope-Creep

Input $\xrightarrow{\text{functionality}}$ Output

Non-functional requirement - "What?" [Except Security]

Every Requirement document should be version controlled

Bug: When the requirement doesn't work

RE:

- (i) Ask for Requirements [Interviews, Observations, JAD, Prototyping]
- (ii) Analyse [Low fidelity: Figma]
- (iii) Negotiate [MSCW]

- Business Analyst $\longleftrightarrow$ Client
- MVP - Minimum Viable Product
 - ↳ What most people agree upon (MUST-HAVES, SHOULD-HAVES)

5/2

→ Requirements:

NEEDS - MVP

WANTS

- MSCW vs. KANO:

Basic features

(Expected to have)

Differentiators

(Non functional over the same functional)

Delighters

(Amazed at a specific feature)

RE:

- (i) Gathering — Reduces Rework
- (ii) Analyse
- (iii) Specification

e.g. Req.: User should be authenticated to buy items on ecommerce site
Good Req. / Bad Req.?

Q) What are the categories of users?

Q) What is authentication?

Assumption: Username & Password

Req.: User should be authenticated with unique username and password to buy items on ecommerce site

[Golden Hammer: Tool for everything]

V & V

↳ Validation: Am I building the product right? [Stakeholders]
↳ Verification: Am I building the right product? [Analysts]

Correctness : Should not defy any laws

Completeness : Given 1 sentence , understand entirely given requirement

Ambiguity : Should be clear

Feasibility : Can it be done ?

Necessary : Is it really needed ?

Prioritized : Categorise into High , Medium & Low

Consistent : Any sentence cannot contradict any other statement .

Concise : Req. should be short & understandable

Traceable : Be able to trace any problem to root cause / root req.
(Avoid multi-level dependency)

Verifiable : Feasible

- Model Driven Development : Without writing code

- Actor initiates behaviour

9/2

SRS → Acceptance

↳ Template

Functional

↑ Direct
↑ Indirect (SYSTEM)

- Problem to Solution ?

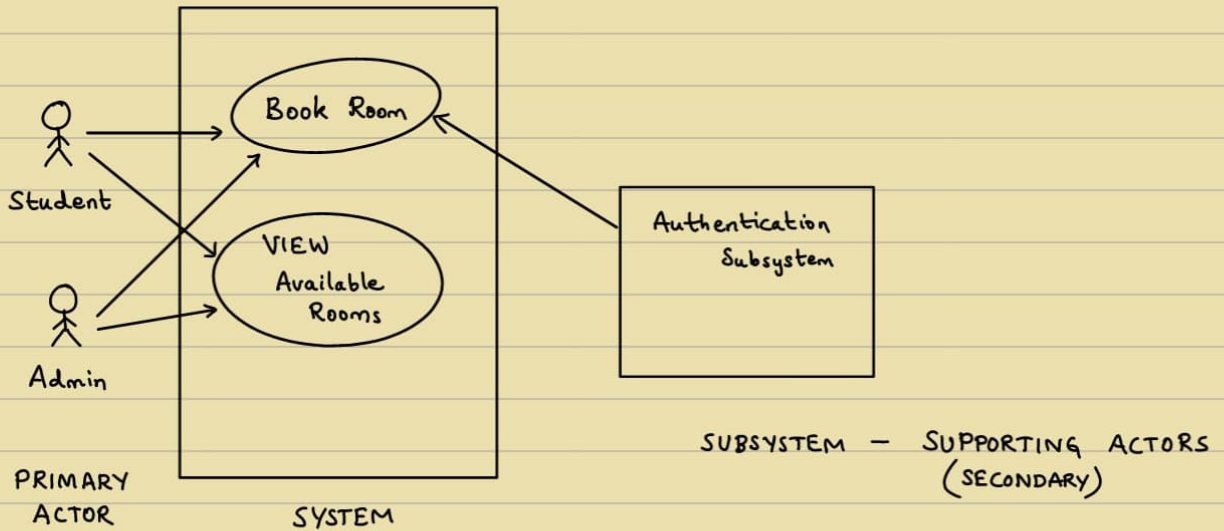
Requirments ⇒ Design

What (Wants) ⇒ How should it do
└─> What must SYSTEM do (Bridge) ─┘

Use Cases / User Stories } Bridging the gap
Conceptual Modeling }

Actor $\Rightarrow$ User story
(Trigger)

Actor : Initiates specific behavior for a system



[No class should have more than 4-5 Methods
No method should have more than 10 LOC]

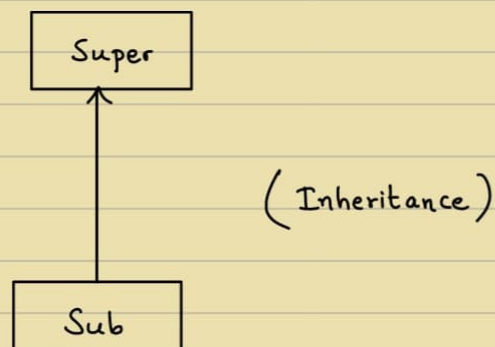
Primary Actor : Initialising

(30:00)

Secondary Actor : Primary use case supporting the system ,
May or may not be used to complete

- Use - Cases : Single functional use
 $\hookrightarrow$ Detailed Description

Post Condition : Implementation of Behaviour



→ Use - case description :

- Use Case Name
- Description (Overview)
- Actor
- Pre - condition
- Main flow
- Alternate flow
- Post - condition

Q) POS :

<u>Actor</u>	<u>Use-case name</u>
Cashier	Process Sales
	Handle Payment
Admin- Manager	Register Cashiers
	Assign cashier to terminal

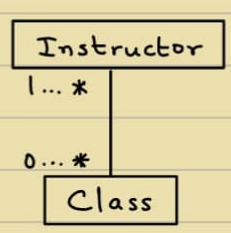
• Use - Case Models $\Rightarrow$ Domain / Conceptual Models $\Rightarrow$ Design Models

- Conceptual concepts : Nouns in Use-case description
Identify Attributes and Relationships
Not Software classes

Private : -
Public : +
Protected : #
/ : Derived

} Access Modifiers

• Association :



19/2

→ Testing :

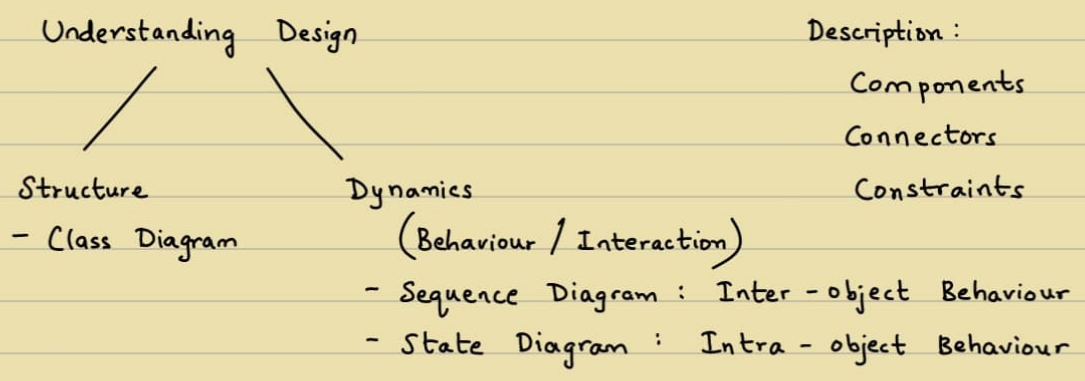
- Class : Group of similar objects
- Exhaustive Testing :
 - (i) Random Testing
 - (ii) Equivalence classes based Testing
 - (iii) Coverage based Testing
- Known bugs
Developers will be separate from Testing.

5/3

Testing : Presence of defects

Good Design - Less no. of defects

- Designing a system :
Don't reinvent the wheel (Scientist vs. Engineer)
UML : Representation of System



Design ≠ Sequence Diagrams
→ Decision Making
Language Agnostic

- Data Abstraction
 - Procedural Abstraction
 - Control Abstraction [TRY-CATCH]
 - ↳ Exception Handling
 - ↳ Iterators
 - ↳ Subroutines
 - Information Hiding
- (Single Responsibility Principle)

Coupling - Interdependence b/w Modules

Good Design :

Loose Coupling

High Cohesion

Bad Design : Tight Coupling / Low Cohesion

∴ We get a single main controller - Large blob

Large class - MOV

Design Smells

• SOLID Principle :

S : Single Responsibility

O : Open / Closed Principle

L : Liskov Substitution (Hierarchy b/w Classes - Subclass, Superclass)

I : Interface Segregation

D : Dependency Inversion

9/3

→ Secure By Design :

→ UI Design & Usability Engg. :

Non-functional req. ← how well they are done

Trade off

UI - Responsive

Intuitive

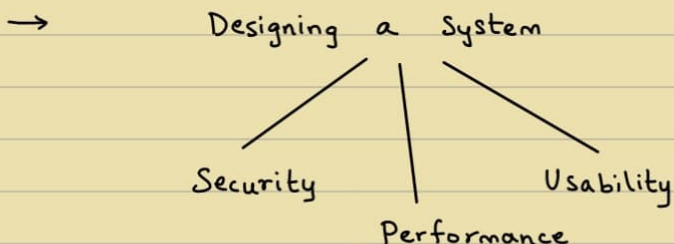
Task Grouping

No Sudden Pop-ups

Right Click (App Message)

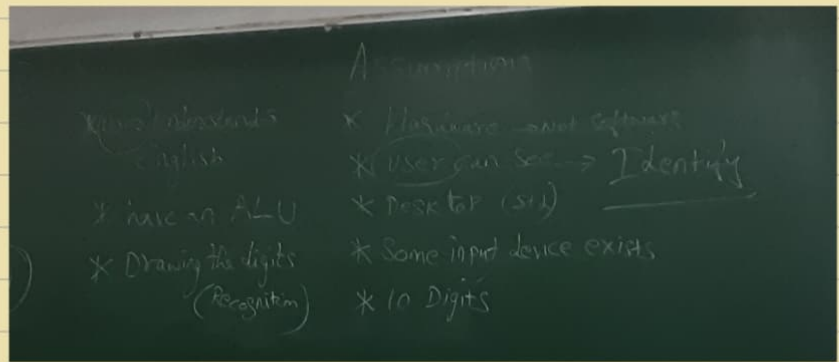
Back from one page to prev.

Navigation



- User Base
- Cognition

UI Hall of Fame
UI Hall of Shame

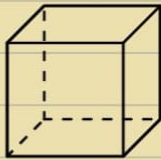


23/3

→ Behaviour Modeling

Object vs. Model of the object

i.e. Language used to communicate matters



Cube

Projection of Cube

Chalk on board

Imperative Language vs. Declarative Language

(Telling how things should be done vs. Telling what to be done)

Is Python declarative or Imperative Language?

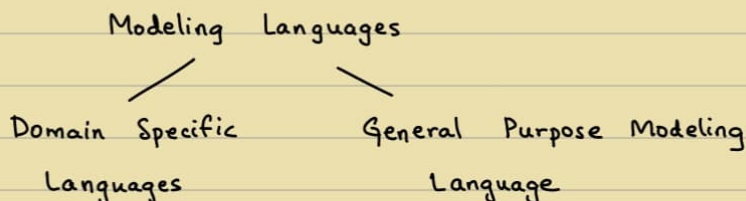
Class Diagrams → UML : Telling what to be done

Sequence diagrams
State diagrams } → Telling how it is to be done

Multiple objects are interacting : Inter - Object Behaviour

∴ Sequence diagrams are used.

Intra - Object Behaviour - State diagrams



- OCL : Object Constraint Language
- VPL
- VHDL : Verilog, Simulink

- BPML : Business Purpose Modeling Language
- Models of Models : Meta Model
Meta Models of Meta Models : Meta Object Facility
- What's Truly Objective Oriented — SMOLTOK

26/3

Web Service

SAAS - Software As A Service

Consuming Service

Supplying Service

SOA - Software Over Arch.

Services → Microservices

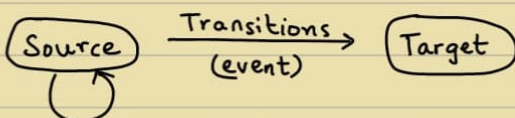
- System Sequence Diagram
Class ↔ Diagram

Interaction Sequence : Top to Bottom
Left to Right

Lifeline : Boxes (Active)

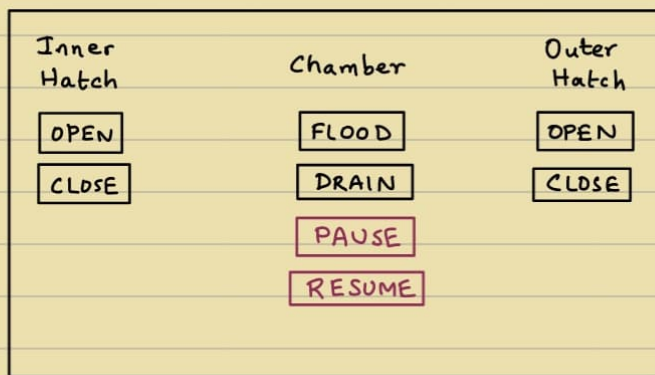
Existence : Dashed

- Currency : Active & Passive (Inactive) Objects
- Three types of messages
 - (i) Synchronous →
 - (ii) Asynchronous →▷
 - (iii) Return ---->
- Concurrency
- alt/par

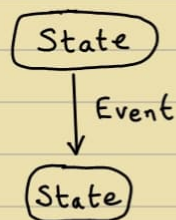


- Business Workflows - Activity Diagram

2/4



Languages : Z, B, ALLOY



Proof $\neq$ Evidence
Show Mathematically

6/4

Problem : Bad food in mess

Sol: Collect regular feedback

Aggregate results & send to vendor

if vendor does not improve, change the vendor

Maintain health checks in between

Change the chefs if no improvements

Discuss budget constraints with the vendor

Check into recipes

Regular Inspection

What is
the problem
(...ilities)



How to address
the problem
(Functional)

FAN-IN

FAN-OUT

Highly dependent : Tightly coupled

Cyclomatic
Complexity

QUALITATIVE (subj.)
vs.
QUANTITATIVE (obj.)

PATTERNS :

(1) GRASP [Atleast 4]
(Loose Coupling)

MVC : Model View Controller

NPA : Non-Performative Asset

Inappropriate Intimacy - Code Smells

9/4

PIP : Performance Improvement Program

16/4

- Software Aging - Everything around it changes
 ↘ Mathematical Product

Fixing a bug : Root-cause Analysis & Impact Analysis
Replication : No Documentation

Refactoring : No new functionality

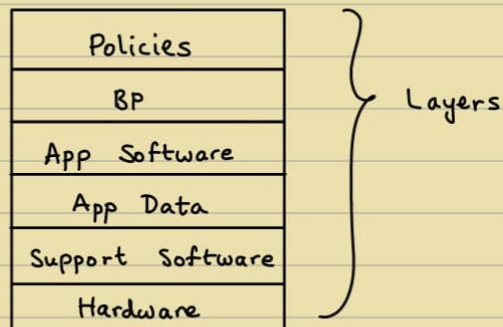
- Smells → Software Evolve

Correctness $\neq$ it works
Correctness = Coverage

Cyclomatic Complexity : Check complexity

Productivity

Business logic & Building functionality - Easy
Data Migration - Hard



Reverse Engineering
+
Forward Engineering
} Round Trip Engineering